COMPTE RENDU DE TRAVAUX PRATIQUES

# Prise en main du logiciel Keil µVision

Microcontrôleur STM32F407 · Projet « Led »

F407

Cortex-M4F

Réalisé par : **Raja Lassioued**
Hibatallah Lafi
Jawaher Jaballah
Talel Jouili

Classe / Groupe : LISI 3 · Groupe C

Date : 22 octobre 2026

# Sommaire

1. Introduction et objectifs . . . . . 3
2. Présentation de Keil µVision (MDK-ARM) . . . . . 3
3. Téléchargement et installation . . . . . 4
4. Présentation de l’interface . . . . . 6
5. Création du projet . . . . . 7
6. Configuration du projet (Options for Target) . . . . . 8
7. Montage matériel . . . . . 9
8. Programme réalisé . . . . . 12
9. Compilation (Build) . . . . . 15
10. Débogage . . . . . 16
11. Chargement dans la carte réelle . . . . . 18
12. Problèmes fréquents et solutions . . . . . 18
13. Conclusion . . . . . 18
Glossaire . . . . . 18

# 1. Introduction et objectifs

Les microcontrôleurs STM32 de la famille Cortex-M sont programmés à l'aide d'un environnement de développement intégré (IDE). **Keil µVision**, édité par Arm, est l'un des plus utilisés dans l'enseignement et l'industrie. Il regroupe dans une seule interface l'éditeur de code, le compilateur C/C++, l'assembleur, le programmateur de la mémoire Flash et le débogueur.

Ce compte rendu présente, captures d'écran à l'appui, la prise en main de Keil µVision à travers un exemple concret : un projet nommé **Led** qui commande trois LED (rouge, orange et verte) branchées sur les broches PA5, PA6 et PA4 du port GPIOA du microcontrôleur **STM32F407VGTx**. Une seconde version du programme, qui fait clignoter deux LED en alternance, est également présentée.

## Objectifs du travail

- Télécharger et installer Keil MDK-ARM, puis vérifier les packs de support du STM32F407.
- Créer un projet, choisir le microcontrôleur et ajouter les composants logiciels (CMSIS, démarrage).
- Écrire un programme en langage C qui configure des broches GPIO par accès direct aux registres.
- Compiler le projet et interpréter le résultat du build.
- Lancer une session de débogage et lire les registres du processeur.

## Démarche suivie




## Matériel et logiciels utilisés

| Élément | Détail |
|---|---|
| Ordinateur | PC sous Windows (dossier utilisateur « a ») |
| IDE | Keil µVision, MDK-ARM version 5.43a (août 2025), installé dans C:\Users\a\AppData\Local\Keil_v5 |
| Compilateur | Arm Compiler V6.24 (armclang) |
| Pack de support | Keil::STM32F4xx_DFP version 2.17.1 (la 3.1.1 est disponible) |
| Composants CMSIS | CMSIS-CORE 6.1.1 et Device Startup 2.6.3 |
| Microcontrôleur | STM32F407VGTx : ARM Cortex-M4, 168 MHz, 192 Ko de RAM, 1 Mo de Flash |
| Carte | STM32F407G-DISC1 (référence MB997E) avec ST-LINK intégré, reliée au PC par un câble USB / mini-USB |
| Montage | Plaque d'essai (breadboard), 3 LED (rouge, orange, verte), résistances de limitation et fils de liaison |

# 2. Présentation de Keil µVision (MDK-ARM)

**MDK (Microcontroller Development Kit)** est la suite logicielle qui contient µVision. Ses composants principaux :

- **µVision IDE** : éditeur, gestionnaire de projet, interface de build et de débogage.
- **Arm Compiler** : compile le code C/C++ en code machine ARM Cortex-M. Ici, la version 6 (V6.24).
- **Device Family Packs (DFP)** : fichiers propres à chaque famille de microcontrôleurs (ici Keil::STM32F4xx_DFP) : fichiers de démarrage, en-têtes CMSIS, algorithmes de programmation Flash.
- **Software Packs / RTE** (Run-Time Environment) : ajout simple de CMSIS, HAL, RTOS, etc.
- **Débogueur et simulateur** : exécution pas à pas, points d'arrêt, observation des registres et des variables.

> **Version d'évaluation**
> La licence gratuite limite la taille du code à 32 Ko. Le message « Running with Code Size Limit: 32K » le rappelle dans la fenêtre Command lors du débogage. Cette limite est largement suffisante pour les travaux pratiques.

## 3. Téléchargement et installation

Le téléchargement se fait depuis le site de Keil (keil.arm.com), rubrique *Download* → *Products*. La page liste les outils disponibles ; pour les microcontrôleurs Cortex-M on choisit **MDK-Arm**, dont la dernière version proposée est la **5.43a (août 2025)**.

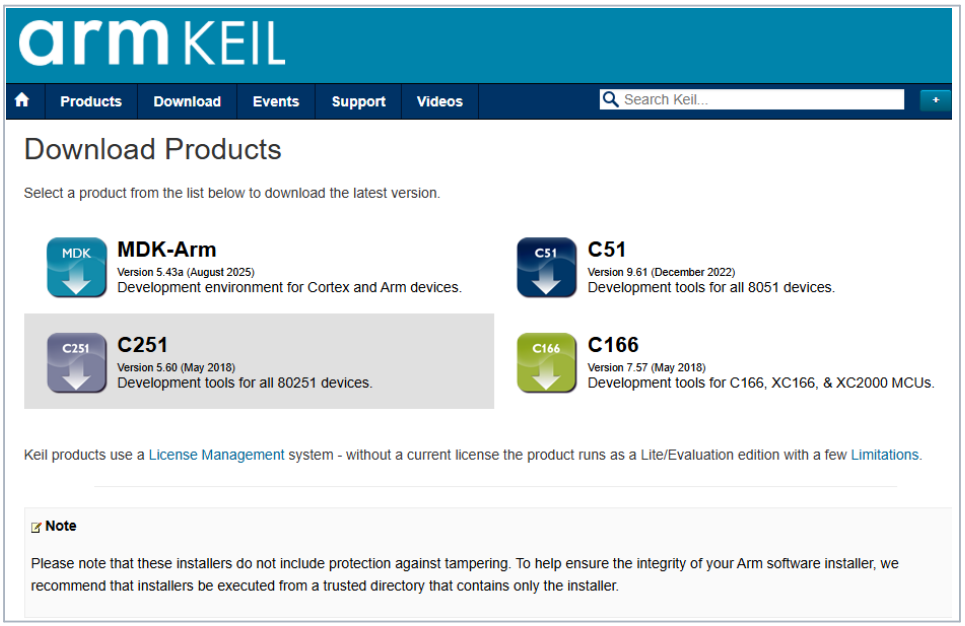



***Figure 1*** *– Page « Download Products » du site Keil : choix de MDK-Arm version 5.43a.*

Après avoir choisi MDK-Arm, une page demande de remplir un formulaire de contact (prénom, nom, e-mail, société, fonction, pays, téléphone). Une note recommande d'exécuter l'installateur depuis un dossier de confiance ne contenant que ce fichier, car l'installateur n'est pas protégé contre la falsification.

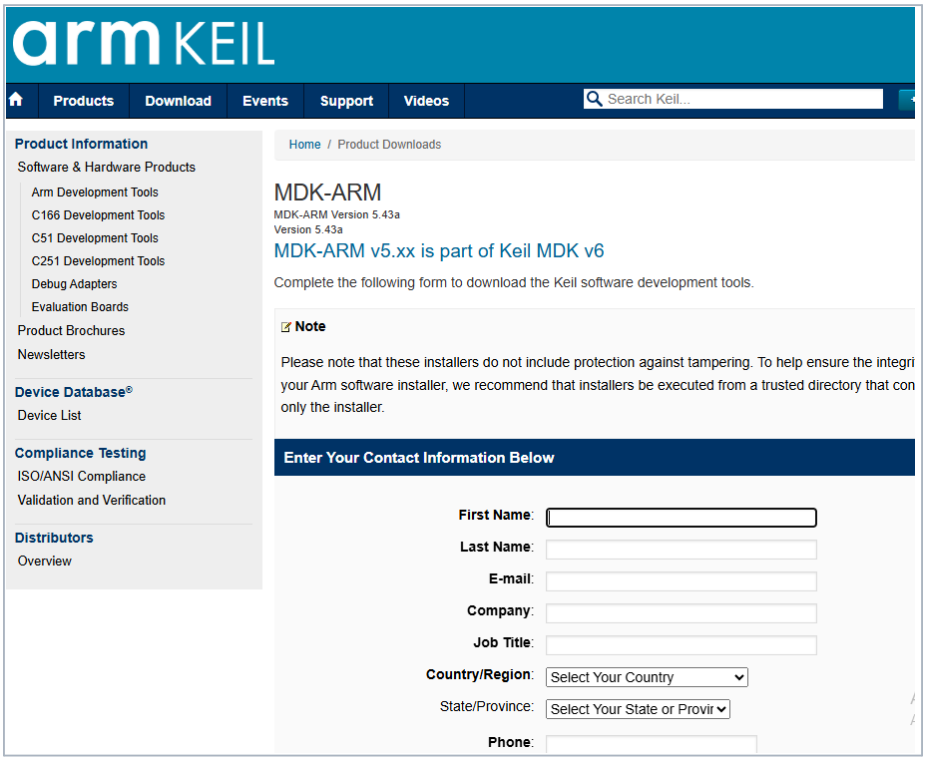



***Figure 2*** *– Formulaire de contact à remplir avant le téléchargement.*

Une fois le formulaire envoyé, la page de téléchargement propose le fichier d'installation **MDK543A.EXE** (environ 850 Mo, daté du 21 août 2025). Elle rappelle aussi les limitations de la version d'évaluation.

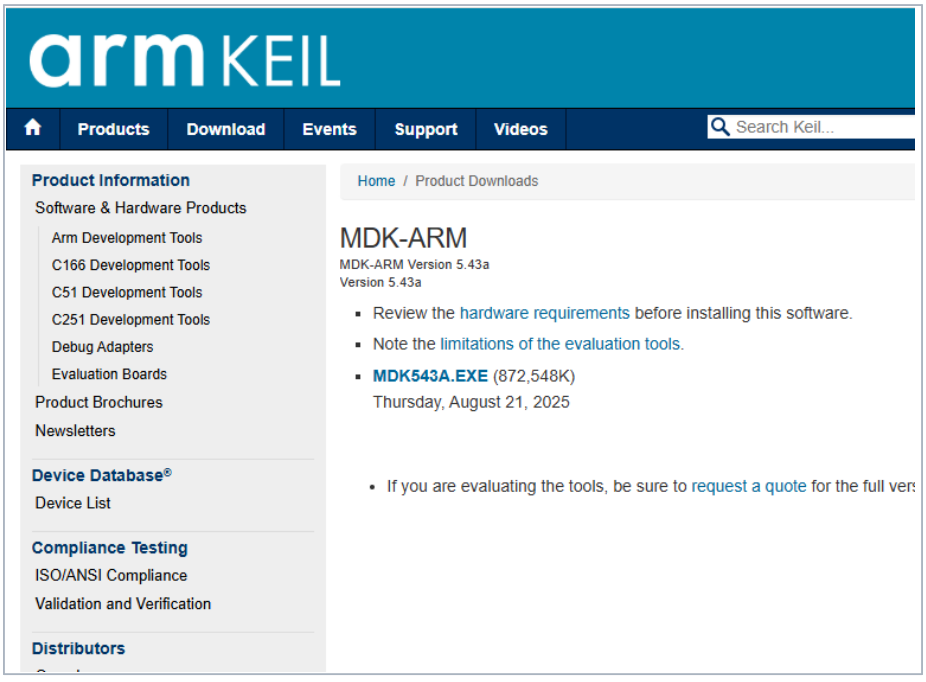



***Figure 3*** *– Page de téléchargement : fichier d'installation MDK543A.EXE.*

On lance ensuite MDK543A.EXE, on accepte la licence et on choisit le dossier d'installation. Dans notre cas, Keil est installé dans le dossier de l'utilisateur (C:\Users\a\AppData\Local\Keil_v5). Au premier démarrage, µVision affiche son écran d'accueil.




***Figure 4** – Écran de démarrage de µVision (Copyright Arm 2013-2025).*

### Vérification des packs avec le Pack Installer

Le **Pack Installer** gère les packs de support des microcontrôleurs. En cherchant « STM32F407V », on retrouve l'arborescence *STMicroelectronics* → *STM32F4 Series* → *STM32F407* → *STM32F407VG* → *STM32F407VGTx*, dont le résumé indique « ARM Cortex-M4, 168 MHz, 192 KiB RAM, 1 MiB ROM ». À droite, le pack **Keil::STM32F4xx_DFP** est installé en version 2.17.1 ; la version 3.1.1 (août 2025) est proposée par le bouton *Update*. Les packs génériques (CMSIS, CMSIS-Compiler, CMSIS-Driver, CMSIS-DSP, CMSIS-NN, CMSIS-RTX, CMSIS-View, MDK-Middleware) sont tous « Up to date ». Les packs sont stockés dans C:\Users\a\AppData\Local\Arm\Packs.

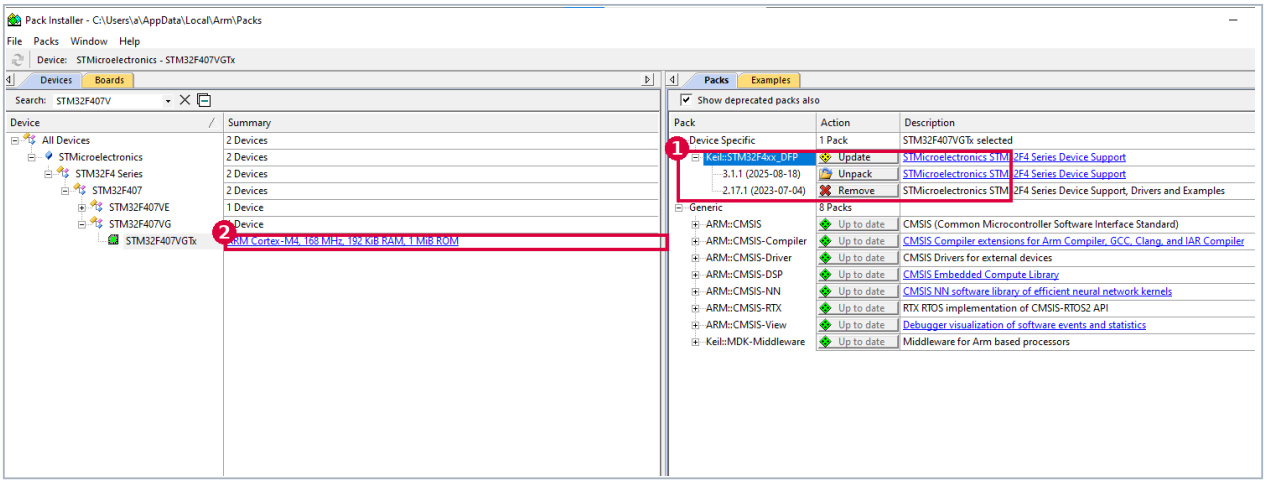



***Figure 5** – Pack Installer : STM32F407VGTx et pack Keil::STM32F4xx_DFP.*

**(1)** Pack de support du STM32F4 : version 2.17.1 installée, version 3.1.1 disponible.
**(2)** Caractéristiques du composant : Cortex-M4, 168 MHz, 192 Ko de RAM, 1 Mo de ROM (Flash).

## 4. Présentation de l'interface

L'interface de µVision, visible sur la figure suivante, comprend :

- **Barre de menus** : File, Edit, View, Project, Flash, Debug, Peripherals, Tools, SVCS, Window, Help.
- **Barres d'outils** : fichiers, navigation, build, chargement Flash (icône Load) et sélecteur de cible *Target 1*.
- **Fenêtre Project** (à gauche) : arborescence du projet.
- **Zone d'édition** (au centre) avec des onglets : ici *led.c* et *startup_stm32f407xx.s*.
- **Fenêtre Build Output** (en bas) : messages du compilateur.

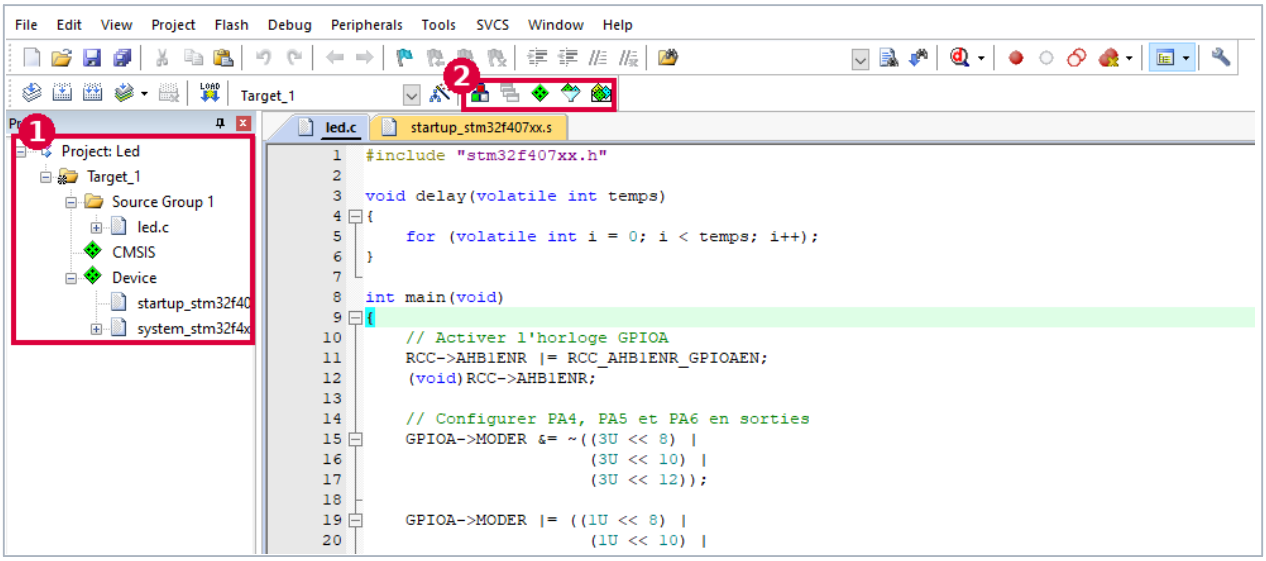



*Figure 6 – Interface de µVision : projet Led, onglets led.c et startup_stm32f407xx.s.*

**(1)** Fenêtre Project : Project: Led → Target 1 → Source Group 1 (led.c), CMSIS, Device (startup et system).
**(2)** Boutons de compilation : Translate, Build, Rebuild, Batch Build et Stop.

### Raccourcis et boutons utiles

| Action | Menu | Raccourci |
|---|---|---|
| Compiler les fichiers modifiés (Build) | Project → Build target | F7 |
| Programmer la Flash (Download) | Flash → Download | F8 |
| Démarrer / arrêter le débogage | Debug → Start/Stop Debug Session | Ctrl + F5 |
| Lancer l'exécution (Run) | Debug → Run | F5 |
| Pas à pas détaillé / principal / sortie | Debug → Step / Step Over / Step Out | F11 / F10 / Ctrl + F11 |
| Point d'arrêt (Breakpoint) | Debug → Insert/Remove Breakpoint | F9 |

## 5. Création du projet

1. Menu **Project → New µVision Project…**, choisir un dossier vide et donner le nom **Led**.
2. Dans *Select Device for Target*, choisir **STMicroelectronics → STM32F4 Series → STM32F407 → STM32F407VGTx**, puis OK.
3. La fenêtre **Manage Run-Time Environment** s'ouvre. On coche **CMSIS → CORE** (version 6.1.1) et **Device → Startup** (version 2.6.3, fournisseur Keil).
4. Après validation, le dossier *Device* contient *startup_stm32f407xx.s* et *system_stm32f4xx.c*.
5. Clic droit sur *Source Group 1* → **Add New Item to Group… → C File (.c)** → nommer **led.c**.

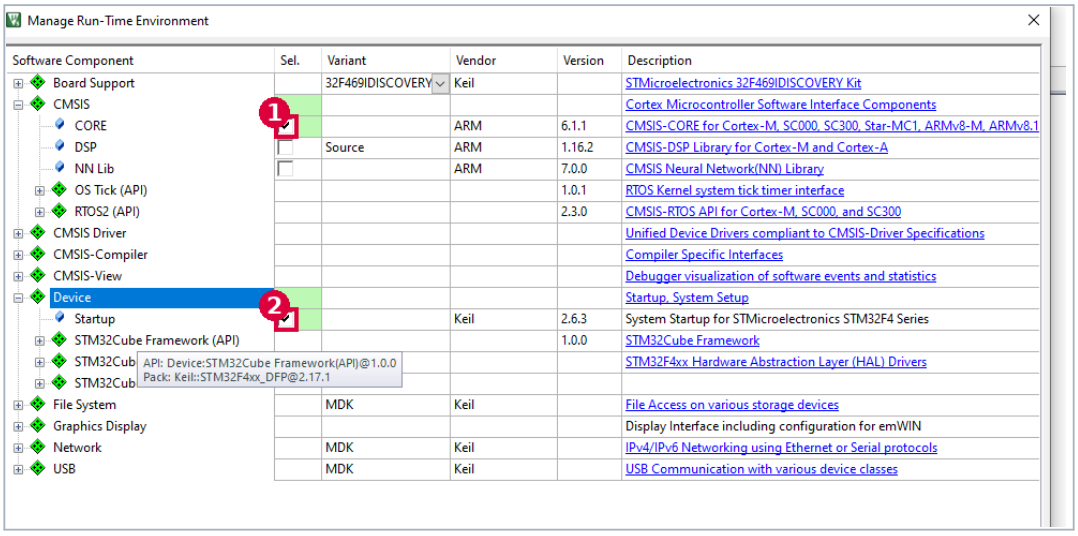



***Figure 7** – Manage Run-Time Environment : CMSIS CORE et Device Startup cochés.*

**(1)** CMSIS → CORE : en-têtes et fonctions de base du cœur Cortex-M (version 6.1.1).

**(2)** Device → Startup : code de démarrage et initialisation système du STM32F4 (version 2.6.3).

> **Attention**
> Dans la fenêtre RTE, la ligne « Board Support » propose par défaut la variante 32F469IDISCOVERY. Elle ne doit pas être cochée : notre microcontrôleur est un STM32F407. L'info-bulle confirme que le pack utilisé est Keil::STM32F4xx_DFP@2.17.1.

# 6. Configuration du projet (Options for Target)

Clic droit sur *Target 1* → **Options for Target** (ou icône Magic Wand). L'onglet **Debug** est divisé en deux colonnes : à gauche le simulateur, à droite le débogueur matériel.

| Réglage | Colonne de gauche : simulateur | Colonne de droite : matériel (ST-Link) |
|---|---|---|
| Mode choisi | **Use Simulator** (sélectionné sur la capture) | **Use: ST-Link Debugger** (carte réelle) |
| Load Application at Startup / Run to main() | Cochés | Cochés |
| CPU / Driver DLL | SARMCM3.DLL, paramètre -REMAP -MPU | SARMCM3.DLL, paramètre -MPU |
| Dialog DLL | DCM.DLL, paramètre -pCM4 | TCM.DLL, paramètre -pCM4 |
| Restore Debug Session Settings | Breakpoints, Toolbox, Watch Windows & Performance Analyzer, Memory Display, System Viewer | Breakpoints, Toolbox, Watch Windows, Tracepoints, Memory Display, System Viewer |

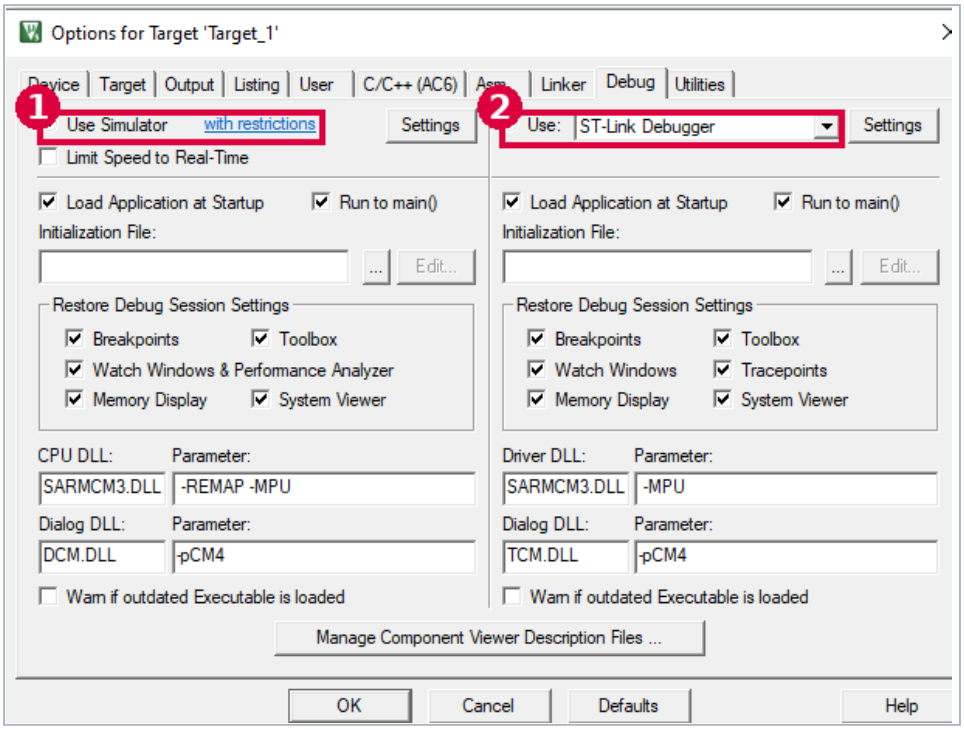



***Figure 8** – Options for Target, onglet Debug : simulateur et ST-Link Debugger.*

**(1)** Use Simulator : le programme s'exécute sur un modèle logiciel du Cortex-M4, sans carte.
**(2)** Use ST-Link Debugger : le programme est chargé et débogué sur la carte réelle.

Le paramètre **-pCM4** indique un cœur Cortex-M4. Pour travailler sur la carte réelle, il faut sélectionner « Use: ST-Link Debugger », puis vérifier dans *Settings* que le ST-Link est détecté et que l'algorithme *STM32F4xx Flash* est présent dans l'onglet Flash Download.

# 7. Montage matériel

Le montage est réalisé sur une plaque d'essai (breadboard) à partir de la carte **STM32F407G-DISC1** (référence MB997E). La carte est alimentée et programmée par un câble USB branché sur le connecteur mini-USB du ST-LINK. Les broches du port GPIOA sont reliées par des fils à la plaque d'essai, où chaque LED est montée en série avec une résistance de limitation de courant.

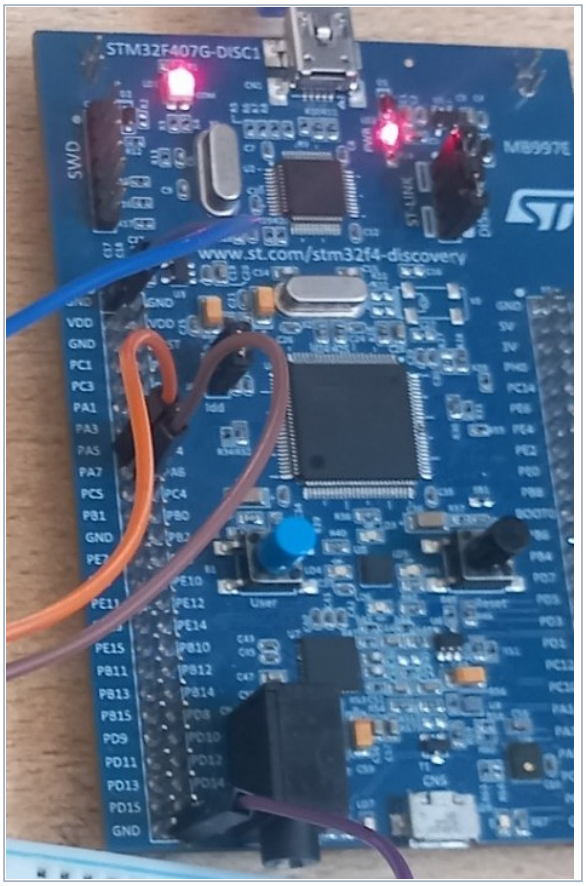



***Figure 9*** *– La carte STM32F407G-DISC1 : microcontrôleur, ST-LINK, boutons USER (bleu) et RESET (noir), connecteur mini-USB et connecteurs d'extension.*

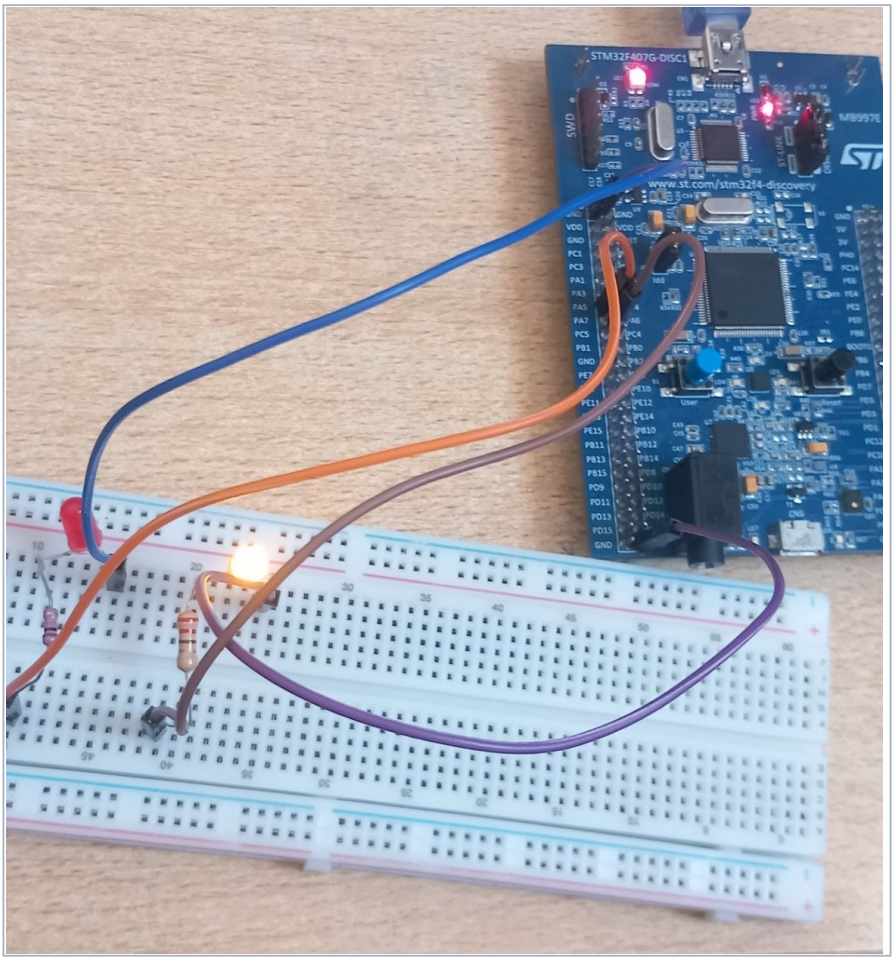



***Figure 10*** *– Montage réalisé : la carte est reliée à la plaque d'essai (LED et résistances) par des fils. Une LED est allumée pendant l'exécution du programme.*

Sur la photo, les voyants rouges de la carte indiquent qu'elle est alimentée et connectée au PC par le ST-LINK.

| Broche | Version 1 : séquence de 3 LED | Version 2 : alternance de 2 LED |
|---|---|---|
| PA4 | LED verte | Non utilisée |
| PA5 | LED rouge | LED1 |
| PA6 | LED orange | LED2 |
| Retour du courant | Masse commune (GND de la carte) | Masse commune (GND de la carte) |

**Calcul de la résistance (exemple indicatif)**
Une sortie du STM32 fournit 3,3 V. Pour une LED rouge dont la tension directe est d'environ 2 V et un courant de 10 mA : R = (3,3 − 2) / 0,010 = 130 Ω. On choisit une valeur normalisée supérieure, par exemple 220 Ω, ce qui donne un courant d'environ 6 mA : la LED et la broche du microcontrôleur sont protégées.

**Remarque**
Les broches PA5, PA6 et PA7 sont aussi reliées au bus SPI1 de l'accéléromètre intégré à la carte. Ici elles sont utilisées comme simples sorties GPIO, ce qui ne pose pas de problème pour ce montage.

# 8. Programme réalisé

Deux programmes ont été écrits en langage C, par accès direct aux registres du microcontrôleur. Dans les deux cas, la démarche est la même : **(1)** activer l'horloge du port GPIOA, **(2)** configurer les broches en sorties, **(3)** choisir le type de sortie, **(4)** commander les LED dans une boucle infinie.

## Version 1 : séquence de trois LED (rouge, orange, vert)

led.c | startup_stm32f407xx.s

```c
#include "stm32f407xx.h"

void delay(volatile int temps)
{
    for (volatile int i = 0; i < temps; i++);
}

int main(void)
{
    // Activer l'horloge GPIOA
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    (void)RCC->AHB1ENR;

    // Configurer PA4, PA5 et PA6 en sorties
    GPIOA->MODER &= ~((3U << 8) |
                      (3U << 10) |
                      (3U << 12));

    GPIOA->MODER |= ((1U << 8) |
                     (1U << 10) |
                     (1U << 12));

    // Sorties push-pull
    GPIOA->OTYPER &= ~((1U << 4) |
                       (1U << 5) |
                       (1U << 6));

    while (1)
    {
        // Rouge ON : PA5
        GPIOA->BSRR = (1U << 5) |
                      (1U << (4 + 16)) |
                      (1U << (6 + 16));
        delay(2000000);

        // Orange ON : PA6
        GPIOA->BSRR = (1U << 6) |
                      (1U << (4 + 16)) |
                      (1U << (5 + 16));
        delay(500000);

        // Vert ON : PA4
```

***Figure 11*** *– led.c (version 1), lignes 1 à 42 : configuration du port et début de la boucle.*

```c
        // Vert ON : PA4
        GPIOA->BSRR = (1U << 4) |
                      (1U << (5 + 16)) |
                      (1U << (6 + 16));
        delay(2000000);
    }
}
```

***Figure 12*** *– led.c (version 1), fin de la boucle principale.*

## Explication de la version 1, bloc par bloc

| Lignes | Code | Explication |
|---|---|---|
| 1 | #include "stm32f407xx.h" | En-tête CMSIS du STM32F407 : il définit les noms des registres (RCC, GPIOA…) et de leurs bits. |
| 3 à 6 | void delay(volatile int temps) | Temporisation logicielle : une boucle vide qui compte jusqu'à *temps*. Le mot *volatile* empêche le compilateur de supprimer la boucle. |
| 8 à 9 | int main(void) { | Point d'entrée du programme, appelé après le code de démarrage. |
| 10 à 12 | RCC->AHB1ENR \|= RCC_AHB1ENR_GPIOAEN; (void)RCC->AHB1ENR; | Active l'horloge du port GPIOA (bit 0 de AHB1ENR). Sans elle, le port est inactif. La relecture du registre garantit que l'horloge est effective avant la suite. |
| 14 à 17 | GPIOA->MODER &= ~((3U << 8) \| (3U << 10) \| (3U << 12)); | Efface les 2 bits de mode des broches PA4 (bits 8-9), PA5 (bits 10-11) et PA6 (bits 12-13). Le masque 3U vaut 11 en binaire. |
| 19 à 21 | GPIOA->MODER \|= ((1U << 8) \| (1U << 10) \| (1U << 12)); | Écrit 01 dans chaque champ : les trois broches passent en mode **sortie**. |
| 23 à 26 | GPIOA->OTYPER &= ~((1U << 4) \| (1U << 5) \| (1U << 6)); | Met les bits 4, 5 et 6 à 0 : sorties de type **push-pull** (la broche impose l'état haut et l'état bas). |
| 28 à 29 | while (1) { | Boucle infinie : le microcontrôleur répète la séquence tant qu'il est alimenté. |
| 30 à 34 | // Rouge ON : PA5 GPIOA->BSRR = … delay(2000000); | Allume la LED rouge (PA5 à 1) et éteint les deux autres, puis attend. Durée la plus longue : 2 000 000 itérations. |
| 36 à 40 | // Orange ON : PA6 GPIOA->BSRR = … delay(500000); | Allume la LED orange (PA6) et éteint les autres. Durée courte : 500 000 itérations, soit 4 fois moins que le rouge. |
| 42 à 46 | // Vert ON : PA4 GPIOA->BSRR = … delay(2000000); | Allume la LED verte (PA4) et éteint les autres. Puis la boucle revient au rouge. |

## Le registre BSRR

Le registre **BSRR** (Bit Set/Reset Register, adresse 0x4002 0018 pour GPIOA) permet de changer l'état de plusieurs broches en **une seule écriture** : les 16 bits de poids faible **mettent à 1** la broche correspondante, les 16 bits de poids fort (position n + 16) **la mettent à 0**. Écrire un 0 n'a aucun effet. L'opération est atomique : contrairement à une modification du registre ODR (lecture, modification, écriture), elle ne peut pas être interrompue en cours de route.

| État | Expression dans le code | Valeur écrite dans BSRR | PA 4 | PA 5 | PA 6 | Durée |
|---|---|---|---|---|---|---|
| Rouge ON | (1U<<5) \| (1U<<(4+16)) \| (1U<<(6+16)) | 0x00500020 | 0 | 1 | 0 | 2 000 000 |
| Orange ON | (1U<<6) \| (1U<<(4+16)) \| (1U<<(5+16)) | 0x00300040 | 0 | 0 | 1 | 500 000 |
| Vert ON | (1U<<4) \| (1U<<(5+16)) \| (1U<<(6+16)) | 0x00600010 | 1 | 0 | 0 | 2 000 000 |

**Rouge ON** → **Orange ON** → **Vert ON** → **Retour au rouge**

**Durée des temporisations :** le microcontrôleur démarre par défaut sur son oscillateur interne HSI de 16 MHz et le code n'est pas optimisé. Une itération de la boucle *delay* dure donc de l'ordre d'une demi-microseconde : 2 000 000 itérations représentent environ une seconde. Cette valeur est approximative ; pour des durées précises, on utilise un timer matériel ou SysTick.

## Version 2 : alternance de deux LED

led.c | startup_stm32f407xx.s

```c
#include "stm32f407xx.h"

int main(void)
{
    /* Activer l'horloge GPIOA */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    (void)RCC->AHB1ENR;

    /* Configurer PA5 et PA6 en sorties */
    GPIOA->MODER &= ~((3U << (5 * 2)) | (3U << (6 * 2)));
    GPIOA->MODER |=  ((1U << (5 * 2)) | (1U << (6 * 2)));

    /* Sorties push-pull, sans pull-up/pull-down */
    GPIOA->OTYPER &= ~((1U << 5) | (1U << 6));
    GPIOA->PUPDR &= ~((3U << (5 * 2)) | (3U << (6 * 2)));

    /* LED1 allumée, LED2 éteinte */
    GPIOA->ODR &= ~((1U << 5) | (1U << 6));
    GPIOA->ODR |= (1U << 5);

    while (1)
    {
        /* Temporisation approximative */
        for (volatile int i = 0; i < 1000000; i++);

        /* Alterner les deux LED */
        GPIOA->ODR ^= ((1U << 5) | (1U << 6));
    }
}
```

***Figure 13*** *– led.c (version 2) : deux LED sur PA5 et PA6 qui s'allument en alternance.*

## Explication de la version 2, bloc par bloc

| Lignes | Code | Explication |
|---|---|---|
| 3 à 4 | int main(void) { | Pas de fonction delay : la temporisation est écrite directement dans la boucle. |
| 5 à 7 | RCC->AHB1ENR \|= RCC_AHB1ENR_GPIOAEN; (void)RCC->AHB1ENR; | Active l'horloge de GPIOA et relit le registre pour confirmer. |
| 9 | GPIOA->MODER &= ~((3U << (5 * 2)) \| (3U << (6 * 2))); | Efface les champs de mode de PA5 et PA6. Écrire (5 * 2) revient à écrire 10 : le bit de départ est 2 fois le numéro de broche. |
| 10 | GPIOA->MODER \|= ((1U << (5 * 2)) \| (1U << (6 * 2))); | Met 01 dans ces champs : PA5 et PA6 sont en sorties. |
| 13 | GPIOA->OTYPER &= ~((1U << 5) \| (1U << 6)); | Sorties push-pull. |
| 14 | GPIOA->PUPDR &= ~((3U << (5 * 2)) \| (3U << (6 * 2))); | Le registre PUPDR gère les résistances de rappel internes. 00 signifie ni pull-up ni pull-down. |

| Lignes | Code | Explication |
|---|---|---|
| 17 à 19 | GPIOA->ODR &= ~((1U << 5) \| (1U << 6));<br>GPIOA->ODR \|= (1U << 5); | État initial : on met d'abord PA5 et PA6 à 0, puis PA5 à 1. La LED1 est allumée et la LED2 éteinte. |
| 21 à 24 | while (1) {<br>for (volatile int i = 0; i < 1000000; i++); | Boucle infinie ; la boucle for vide crée une temporisation d'environ une demi-seconde. |
| 26 à 27 | GPIOA->ODR ^= ((1U << 5) \| (1U << 6)); | L'opérateur XOR (^=) **inverse** les bits 5 et 6 : la LED allumée s'éteint et la LED éteinte s'allume. |

| Étape | PA5 (LED1) | PA6 (LED2) |
|---|---|---|
| Initialisation | 1 (allumée) | 0 (éteinte) |
| Après le 1er XOR | 0 (éteinte) | 1 (allumée) |
| Après le 2e XOR | 1 (allumée) | 0 (éteinte) |
| Et ainsi de suite… | alternance | alternance |

### Comparaison des deux versions

| Critère | Version 1 | Version 2 |
|---|---|---|
| Nombre de LED | 3 (PA4, PA5, PA6) | 2 (PA5, PA6) |
| Registre de commande | BSRR (set/reset atomique) | ODR avec opérateur XOR (inversion) |
| Comportement | Séquence rouge, orange, vert avec durées différentes | Alternance régulière de deux LED |
| Temporisation | Fonction delay() réutilisable | Boucle for écrite dans le programme |
| Configuration des résistances | Non précisée (état par défaut) | PUPDR mis à 00 (sans pull-up ni pull-down) |

## 9. Compilation (Build)

Les étapes de compilation et de débogage présentées ci-dessous concernent la version 1 du programme. La compilation se lance avec **Build (F7)** ; **Rebuild** recompile tout le projet. Le résultat s'affiche dans la fenêtre *Build Output*.

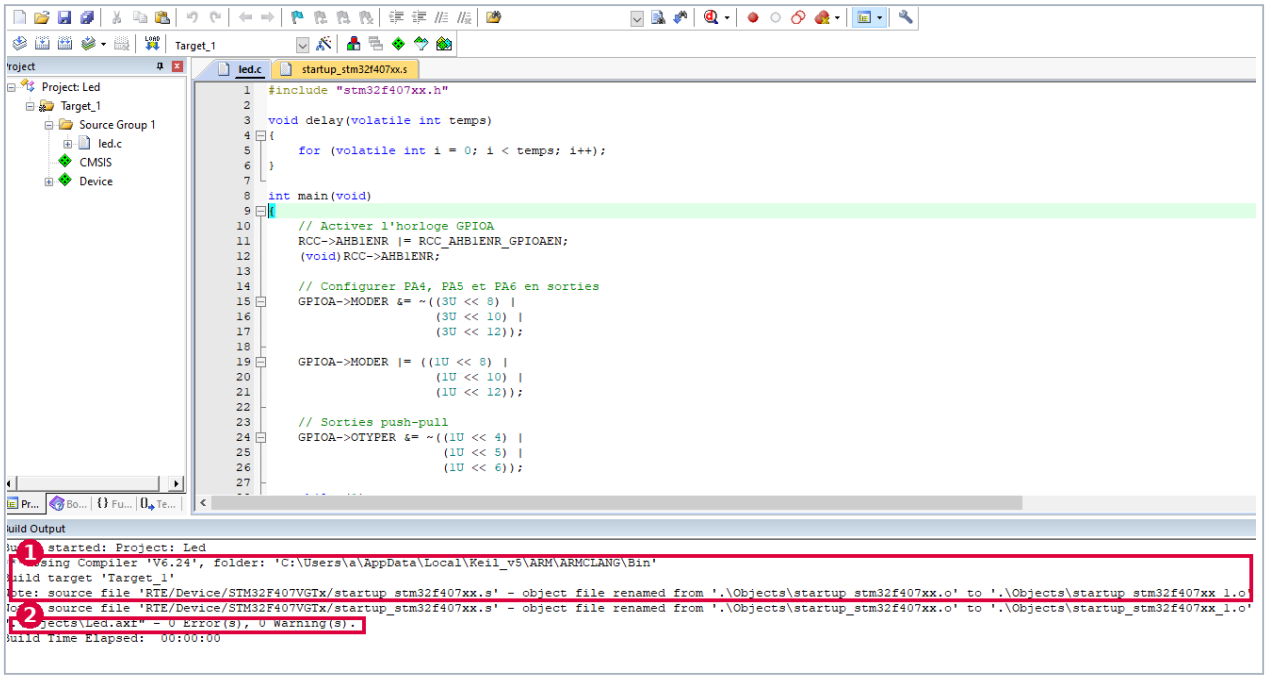



***Figure 14** – Build Output : compilation réussie, 0 erreur et 0 avertissement.*

**(1)** Compilateur utilisé : Arm Compiler V6.24 ; cible Target 1 ; le fichier objet du démarrage est renommé startup_stm32f407xx_1.o pour éviter un conflit de nom (simple note, pas une erreur).
**(2)** Résultat : Led.axf – 0 Error(s), 0 Warning(s). Temps de compilation : 00:00:00.

Le fichier **Led.axf** est l'exécutable produit par l'éditeur de liens. C'est lui que le débogueur charge dans la mémoire du microcontrôleur (ou du simulateur).

# 10. Débogage

Le débogueur se lance avec **Debug → Start/Stop Debug Session** (Ctrl+F5). Ici la session utilise le simulateur : la fenêtre Command affiche « Running with Code Size Limit: 32K » puis le chargement de *Led.axf*. L'exécution s'arrête au début de *main*, comme l'indique la flèche jaune sur l'accolade ouvrante (ligne 9).

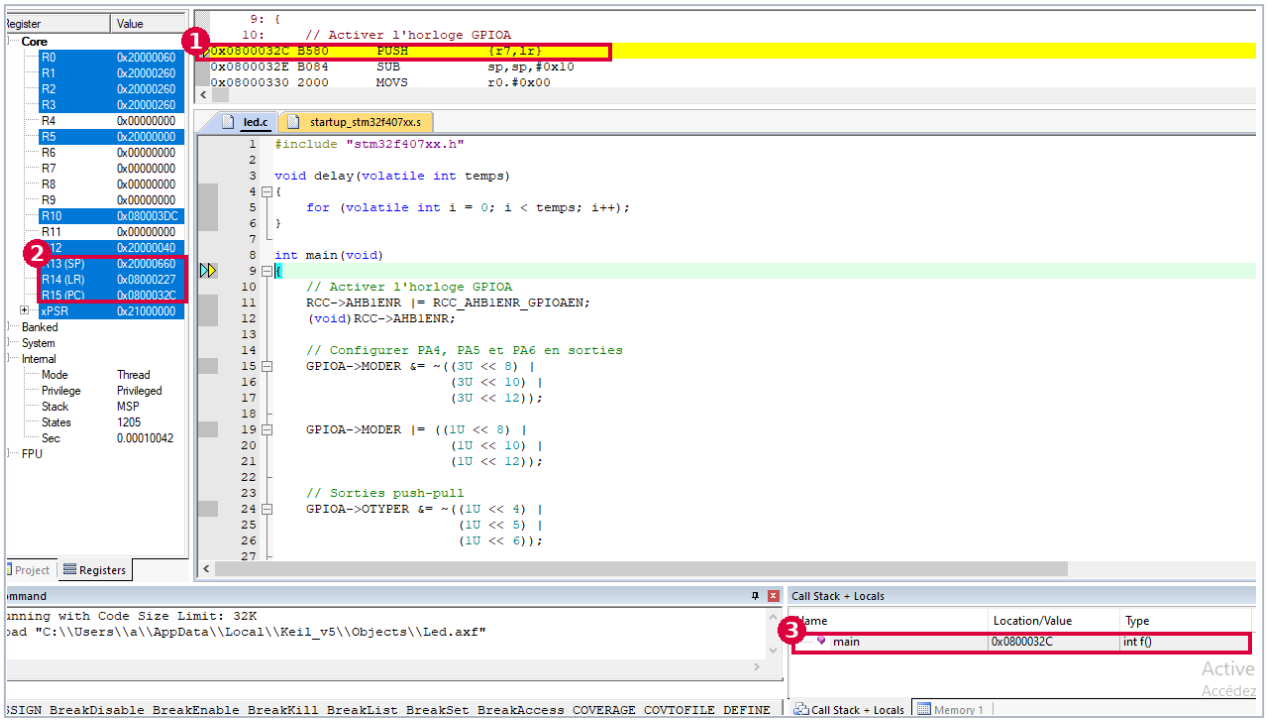



***Figure 15** – Session de débogage : registres du cœur, désassemblage, éditeur et pile d'appels.*

**(1)** Fenêtre de désassemblage : première instruction de main, PUSH {r7,lr}, à l'adresse 0x0800032C.

**(2)** Registres R13 (SP), R14 (LR) et R15 (PC) du cœur.

**(3)** Pile d'appels (Call Stack) : la fonction main est en cours d'exécution.

## Lecture de la fenêtre Registers

| Registre | Valeur | Signification |
|---|---|---|
| R15 (PC) | 0x0800032C | Compteur de programme : adresse, en Flash (0x0800 0000), de la prochaine instruction |
| R13 (SP) | 0x20000660 | Pointeur de pile, situé en SRAM (0x2000 0000) |
| R14 (LR) | 0x08000227 | Registre de lien : adresse de retour (bit 0 à 1 = mode Thumb) |
| xPSR | 0x21000000 | Registre d'état : bit T (Thumb) à 1 et drapeau C (retenue) à 1 |
| Mode / Privilege / Stack | Thread / Privileged / MSP | Exécution normale du programme, avec la pile principale |
| States / Sec | 1205 / 0,00010042 s | Cycles et temps simulés depuis le reset |

**À retenir**
Le programme est stocké en **Flash** (adresses 0x0800 xxxx) et la pile se trouve en **SRAM** (adresses 0x2000 xxxx). Cette séparation se lit directement dans les registres PC et SP.

## Outils du débogueur

- **Run (F5)** et **Stop** : lancer ou arrêter l'exécution.
- **Step Into (F11)**, **Step Over (F10)**, **Step Out (Ctrl+F11)** : exécution pas à pas.
- **Breakpoint (F9)** : arrêt de l'exécution sur une ligne choisie.
- **Registers**, **Watch** et **Memory** : observation des registres, des variables et de la mémoire.

- **Peripherals → GPIOA** : visualisation des registres MODER, OTYPER et ODR, dont les bits changent pendant l'exécution.

## 11. Chargement dans la carte réelle

1. Brancher la carte au PC par câble **mini-USB** (connecteur du ST-LINK).
2. Dans *Options for Target → Debug*, choisir **ST-Link Debugger**.
3. Menu **Flash → Download** (ou icône Load, touche **F8**) : les messages « Programming Done » puis « Verify OK » confirment l'écriture en Flash.
4. Appuyer sur le bouton **RESET** de la carte pour lancer le programme.

Le résultat est visible sur la photo du montage (section 7) : la carte, alimentée par le câble USB, commande les LED de la plaque d'essai.

## 12. Problèmes fréquents et solutions

| Problème | Cause probable | Solution |
|---|---|---|
| No ST-Link detected | Pilote absent ou câble défectueux | Installer le pilote ST-Link, changer de câble ou de port USB |
| Cannot access Target | Mauvais réglage Debug ou carte verrouillée | Vérifier Debug → ST-Link, mettre Connect: under Reset |
| Cannot open source input file | Pack ou RTE non coché | Cocher CMSIS CORE et Device Startup |
| Undefined symbol (L6218E) | Fichier manquant dans le projet | Vérifier que led.c et le fichier de démarrage sont dans le projet |
| Flash Download failed | Algorithme Flash incorrect | Utiliser « STM32F4xx 1MB Flash » dans Flash Download |
| Code supérieur à 32 Ko refusé | Limite de la version d'évaluation | Réduire la taille du code ou utiliser une licence |

## 13. Conclusion

Ce travail a permis de parcourir toute la chaîne de développement avec Keil µVision : téléchargement et installation de MDK-ARM 5.43a, vérification des packs, création du projet *Led* pour le STM32F407VGTx, configuration, compilation sans erreur avec Arm Compiler 6 et débogage avec observation des registres du cœur. Deux programmes ont été réalisés : une séquence de trois LED (rouge, orange, vert) commandée par le registre BSRR, et une alternance de deux LED obtenue par inversion du registre ODR. Dans les deux cas, les broches du port GPIOA sont configurées en sorties push-pull par accès direct aux registres RCC et GPIOA, ce qui permet de comprendre le fonctionnement matériel du microcontrôleur. Le montage sur plaque d'essai avec la carte STM32F407G-DISC1 confirme le bon fonctionnement du programme.

## Glossaire

| Terme | Définition |
|---|---|
| CMSIS | Interface logicielle standard d'Arm pour les microcontrôleurs Cortex-M |

| Terme | Définition |
|---|---|
| DFP | Device Family Pack : pack de support d’une famille de microcontrôleurs |
| RTE | Run-Time Environment : gestion des composants logiciels du projet |
| GPIO | General Purpose Input/Output : broche d’entrée/sortie à usage général |
| Push-pull | Sortie qui impose activement l’état haut et l’état bas |
| BSRR | Bit Set/Reset Register : registre qui met des broches à 1 ou à 0 en une écriture atomique |
| ODR / PUPDR | Registre de sortie des broches / registre des résistances de rappel (pull-up, pull-down) |
| XOR (^) | Opération qui inverse les bits dont le masque vaut 1 |
| Breadboard | Plaque d’essai pour câbler un montage sans soudure |
| AXF | Fichier exécutable produit par l’éditeur de liens, utilisé par le débogueur |
| SWD / ST-LINK | Liaison de programmation et de débogage à 2 fils / interface matérielle de ST |

CARTE DE DÉVELOPPEMENT

# STM32F407 Discovery

F407

Cortex-M4F

Présentation du microcontrôleur et de la carte de démonstration de STMicroelectronics

Systèmes embarqués · octobre 2026

# Une carte de développement complète

**168 MHz**

Fréquence maximale du cœur

**1 Mo**

Mémoire Flash pour le programme

**192 Ko**

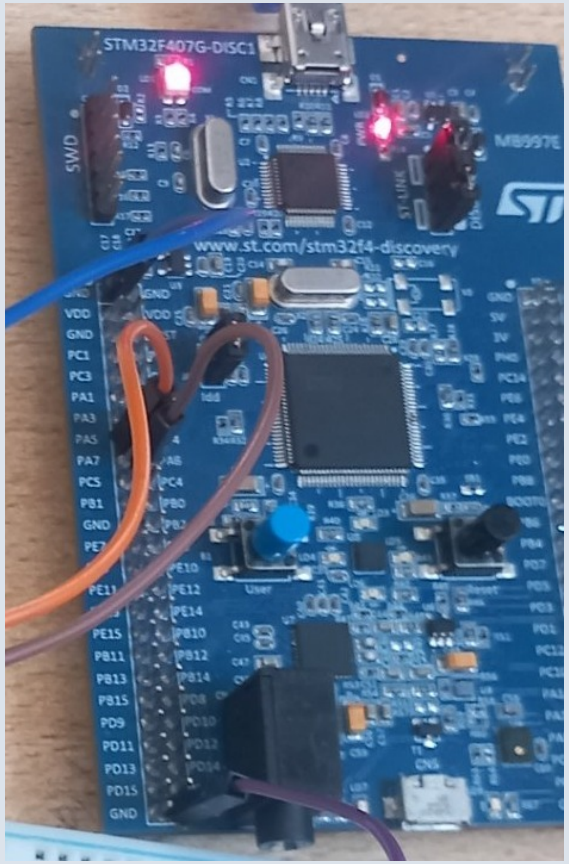
Mémoire SRAM pour les données

**32 bits**

Cœur ARM Cortex-M4 avec FPU et instructions DSP

La STM32F407 Discovery associe un microcontrôleur performant à un programmateur intégré, des capteurs et une sortie audio.

# La carte STM32F407G-DISC1




- Microcontrôleur STM32F407VGT6
- ST-LINK intégré, relié au PC par le connecteur mini-USB
- Bouton USER (bleu) et bouton RESET (noir)
- Connecteur SWD et deux rangées d’extension
- Référence de la carte : STM32F407G-DISC1 (MB997E)

# Le microcontrôleur STM32F407VGT6

| Caractéristique | Valeur |
| --- | --- |
| Cœur | ARM Cortex-M4 32 bits avec FPU simple précision |
| Fréquence maximale | 168 MHz (environ 210 DMIPS) |
| Mémoire Flash | 1 Mo |
| Mémoire SRAM | 192 Ko, dont 64 Ko de CCM |
| Tension d’alimentation | 1,8 V à 3,6 V |
| Boîtier | LQFP100, jusqu’à 82 entrées/sorties |

# Des périphériques très riches

### Entrées/sorties

Jusqu’à 82 broches GPIO avec interruptions externes

### Timers

17 timers, dont des timers avancés pour la MLI moteur

### Analogique

3 convertisseurs ADC et 2 DAC, tous en 12 bits

### Liaisons série

3 I²C, 4 USART et 2 UART, 3 SPI, 2 CAN

### USB et réseau

USB OTG pleine et haute vitesse, MAC Ethernet

### Mémoires et image

SDIO, FSMC pour mémoires externes, interface caméra DCMI

# Les composants de la carte

### ST-LINK/V2

Programmateur et débogueur intégré à la carte

### Accéléromètre

Capteur MEMS 3 axes relié en SPI

### Microphone

Microphone MEMS numérique MP45DT02

### Audio

DAC CS43L22 avec amplificateur classe D et prise jack

### LED et boutons

4 LED utilisateur, un bouton USER et un bouton RESET

### Connecteurs

USB OTG micro-AB et deux rangées d’extension P1 et P2

# Les broches utiles

| Fonction | Broches | Détail |
| --- | --- | --- |
| LED verte, orange, rouge, bleue | PD12, PD13, PD14, PD15 | Sorties vers les LED |
| Bouton USER | PA0 | Entrée numérique |
| Accéléromètre | PA5, PA6, PA7, PE3 | SPI1 et chip select |
| Audio CS43L22 | PB6, PB9 | I²C1 pour la commande |
| USB OTG FS | PA11, PA12 | Lignes D− et D+ |
| Programmation SWD | PA13, PA14 | SWDIO et SWCLK |

# Alimentation et horloges

## Alimentation

- 5 V par le câble mini-USB du ST-LINK
- Régulateur qui fournit le 3,3 V du microcontrôleur
- Broches 3V, 5V et GND sur les connecteurs

## Horloges

- Quartz externe HSE de 8 MHz
- PLL pour monter jusqu’à 168 MHz
- Oscillateur interne HSI de 16 MHz utilisé par défaut au démarrage

# Programmation et débogage

**1**

### Connecter

Un câble mini-USB relie le ST-LINK au PC

**2**

### Programmer

Keil µVision, STM32CubeIDE ou IAR chargent le code par SWD

**3**

### Déboguer

Points d’arrêt, pas à pas, registres et variables en direct

Le ST-LINK intégré évite d’acheter un programmateur externe.

# Applications typiques

## Traitement du signal

Filtres et audio grâce au FPU et aux instructions DSP

## Commande de moteurs

Signaux MLI générés par les timers avancés

## Acquisition de mesures

Capteurs lus par les ADC, transferts rapides par DMA

## Systèmes communicants

Échanges par USB, CAN, SPI, I²C et Ethernet

# En résumé

- Un cœur Cortex-M4F rapide : 168 MHz, FPU et DSP
- Une carte prête à l’emploi : ST-LINK, capteurs, audio et LED
- Un outil idéal pour apprendre, ici avec Keil µVision